

AI Engineering & Model Development Track

Programme Delivery Details & Engagement Framework

Confidential — For Internal Business Review Only

1. Context & Purpose

EXL is investing in a large-scale capability building initiative to ready its technology workforce for client-facing AI deployments across Banking, Financial Services, Insurance (BFSI), and Retail. As EXL's project pipeline increasingly demands engineers who can build, deploy, and operate production-grade AI systems, the gap between conventional software training and real-world AI engineering competency must be closed with precision.

This document accompanies the proposed AI Engineering & Model Development Track — a 35-hour Instructor-Led Training (ILT) programme covering the complete spectrum from LLM integration and RAG systems to fine-tuning, agentic workflows, multimodal AI, and LLMOps. The programme has been designed from the ground up for practitioners who will be deployed on live client engagements immediately upon completion.

The following sections provide business stakeholders and programme owners with detailed visibility into: how each module will be delivered and what participants will actually build; the domain-specific context embedded in every session; how daily performance will be assessed; how participant engagement will be monitored and enforced; and the capstone project options available.

Every example, assignment, dataset, and capstone project in this programme is grounded in BFSI and Retail domain realities. Participants will not work on toy problems. They will build systems they will encounter on real engagements — loan processing agents, fraud detection pipelines, regulatory RAG systems, insurance claims platforms, and multimodal document processors.

2. Programme Structure at a Glance

#	Module	Core Skill Outcome	Hours
0	Pre-Readings with LMS (Self-Paced)	Python, NumPy, ML basics, API calls	3 hrs
1	Python & Engineering Foundations for AI Systems	Async APIs, Pydantic, FastAPI, LLM SDKs	3.5 hrs
2	Machine Learning Engineering — Applied Production Skills	Pipelines, MLflow, Scikit-learn, REST serving	3.5 hrs
3	Deep Learning & Neural Network Engineering	PyTorch, Transformers, GPU training, ONNX	3.5 hrs
4	LLM Engineering — Working with Foundation Models	APIs, function calling, routing, cost tracking	3.5 hrs
5	RAG Systems — Build and Optimise End-to-End	Chunking, embeddings, hybrid search, RAGAS	3.5 hrs
6	Fine-Tuning, Alignment & Model Customisation	LoRA, QLoRA, SFT, DPO, LLM-as-Judge	3.5 hrs
7	AI Agent Engineering	LangGraph, multi-agent, tools, checkpointing	3.5 hrs
8	Multimodal AI Engineering	Vision-LLMs, Whisper, document AI pipelines	3.5 hrs
9	LLMOps — Deploying and Operating AI in Production	vLLM, Docker/K8s, CI/CD, observability	3.5 hrs
10	AI Evaluation, Safety & Responsible Engineering	Red-teaming, guardrails, bias, hallucination	3.5 hrs
—	Capstone Project & Shark Tank Presentation	End-to-end industry project	Post-ILT

3. Delivery Approach & Pedagogy

The programme follows a Build-to-Deploy methodology. Every session is structured around a real-world industry scenario that participants must solve by the end of that session. Theory is introduced only in the context of solving the problem at hand. Each module session runs for approximately 3.5 hours (210 minutes), structured as follows:

How Each Session Is Structured — 3.5-Hour Module Plan

- **Scenario Briefing & Business Context (15 min):** A realistic BFSI or Retail problem is presented with full domain context — what business pain exists, what the downstream impact is, and why the technical approach matters. Participants understand the problem before writing a single line of code.
 - **Concept Introduction & Live Demo (30 min):** The instructor introduces the technical concept required to solve the problem. Delivery is demonstration-first — the instructor codes a minimal working example live, explaining design decisions as they go. No slides-only explanations.
 - **Break (10 min)**
 - **Guided Build — Part 1: Core Implementation (50 min):** The instructor and participants build the primary solution together in a live coding environment. Every component is tied back to the scenario. Participants code along; the instructor pauses at key decision points and invites participants to suggest the next step before revealing the approach.
 - **Guided Build — Part 2: Integration, Edge Cases & Testing (40 min):** The solution is extended to handle production-grade concerns — error handling, validation, logging, and at least one domain-specific edge case (e.g., what happens when a fraud model receives a malformed transaction? When an LLM returns an invalid schema?). Basic tests are written.
 - **Break (10 min)**
 - **Independent Extension Exercise (30 min):** A variation of the problem is introduced — participants work independently to extend or adapt what was built. This is the primary check that participants understand the concept, not just the code they copied. The instructor circulates and calls on participants to explain their approach.
 - **Group Debrief & Peer Code Review (15 min):** 2–3 participants share their extension. The group identifies what worked, what could be improved, and how the approach would scale. Peer review builds the code review culture expected on enterprise teams.
 - **Production Considerations & Q&A (10 min):** The instructor closes by discussing what changes when this system goes to production at scale — performance, compliance, cost governance, monitoring. Open Q&A.
- Total session time: 210 minutes.

Domain Embedding Principles

- All datasets used are domain-specific: loan applications, insurance claims, transaction data, regulatory documents, customer service logs.
- Every tool and library is taught in the context of BFSI/Retail use cases — not generic tutorials.
- Instructors bring live examples from their own delivery experience in banking, insurance, and retail AI projects.
- Compliance, auditability, and explainability requirements of regulated industries are woven into technical instruction — not treated as add-ons.

4. Module-by-Module: Session Content, Assignments & Evaluation

The following sections detail two sample sessions for each module, along with the module assignment and evaluation rubric. All scenarios are drawn from Banking, Financial Services, Insurance (BFSI), and Retail domains.

Module 1: Python & Engineering Foundations for AI Systems

This module establishes the engineering backbone for every AI system participants will build in the programme. The focus is not on Python syntax — it is on production-quality code structure, async patterns for high-throughput AI workloads, strict data validation, and the codebase disciplines required in regulated environments. BFSI clients expect deployable code on day one, not prototype notebooks.

Sample Session 1 — Async Batch Transaction Categorisation Service

Business Scenario	A large retail bank processes 50,000+ daily transactions that need to be automatically categorised (Groceries, Utilities, EMI, International Transfer, Insurance, etc.) for personal finance dashboards and AML pre-screening. A synchronous approach would take 14+ hours; an async architecture reduces this to under 30 minutes.
What Participants Build	An async FastAPI service that accepts batches of up to 100 transaction descriptions, calls an LLM API concurrently using <code>asyncio.gather()</code> , enforces strict Pydantic schemas (category must be from a predefined enum, amount must be a positive float), tracks per-call latency and token consumption, implements exponential backoff for rate limit errors, and returns a batch cost summary (total tokens, estimated USD cost, avg latency ms).
Key Takeaways	• Why async is not optional at scale: a 50,000-transaction daily volume makes synchronous LLM calls operationally infeasible. • Pydantic schemas as a first line of defence — enforcing business rules (valid categories, numeric types) at the API boundary prevents downstream data quality issues. • Token budgeting and cost-per-call modelling before going to production avoids bill shock in live environments.

Sample Session 2 — Codebase Architecture for Regulated AI Environments

Business Scenario	BFSI regulators can require that a firm demonstrate exactly what code produced a given model output. Poorly structured, undocumented repositories are a compliance risk. Engineers joining projects at banks or insurers must immediately operate within enterprise-grade code standards.
What Participants Build	A reference AI codebase structure with clear separation of config, API layer, model logic, prompt templates, and test harness. Secrets management using <code>python-dotenv</code> and environment variable injection — zero hardcoded credentials. Dependency pinning in <code>requirements.txt</code> for reproducibility. Git branching strategy (feature → dev → staging → prod with protected main). A PR checklist tailored to AI code review including prompt version logging, schema change documentation, and test coverage requirements.
Key Takeaways	• Codebase structure is not a style preference in BFSI — it is an audit requirement.

- Dev/staging/prod environment parity prevents the "works on my machine" failure mode in regulated deployments.
- Pinned dependencies and documented prompt versions are the minimum required for AI system reproducibility.

Module Assignment — Production-Ready Email Routing Inference API

Business Scenario	An insurance company receives 3,000+ customer emails daily. These must be routed to the correct team — New Claim, Claim Update, Policy Inquiry, Complaint, or Renewal — within 2 minutes of receipt. The current manual triage team of 5 is a bottleneck.
Deliverables	• Async FastAPI service accepting batches of up to 50 emails; demonstrate throughput vs. sequential baseline. • Strict Pydantic schemas: category must be one of 5 valid enums, confidence score must be between 0 and 1. • Concurrent LLM calls using <code>asyncio.gather()</code> with configurable concurrency limits. • Per-request structured logging: timestamp, input length (chars), predicted category, confidence, latency (ms), tokens used, estimated cost in USD. • Error handling: retry with exponential backoff on rate limits; fallback to "Unclassified" after 3 failures. • Unit tests for schema validation, error handling, and at least 10 email classification test cases. • Clean GitHub repository: proper folder structure, <code>.env.example</code>, README with setup instructions, pinned requirements.txt.
Tools & Stack	Python, FastAPI, asyncio, Pydantic v2, OpenAI/Anthropic SDK, pytest, python-dotenv, Git
Evaluation Criteria	• Async implementation correctness and throughput improvement (25%) • Pydantic schema strictness and error handling completeness (25%) • Logging structure: all required fields present and accurate (20%) • Code architecture: modularity, separation of concerns, clean repo structure (20%) • Test coverage and quality (10%)

Module 2: Machine Learning Engineering — Applied Production Skills

This module focuses on classical ML engineering as a production discipline, not academic exercise. In BFSI, classical ML remains the backbone of credit scoring, fraud detection, and risk modelling. Participants learn to build full pipelines that are experiment-tracked, reproducible, auditable, and served via REST APIs — exactly as expected on client engagements.

Sample Session 1 — Credit Card Fraud Detection Pipeline with Class Imbalance Handling

Business Scenario	A payment network processes 2 million daily transactions. Even a 0.01% miss rate means 200 fraudulent transactions per day. The model must be optimised for precision-recall trade-offs, not accuracy, and all experiment decisions must be defensible to the risk team.
What Participants Build	A full scikit-learn pipeline on the IEEE-CIS Fraud Detection dataset. Feature engineering: velocity features (transactions per hour per card), device fingerprint features, merchant category encoding, time-of-day cyclical encoding using sin/cos. Class imbalance handling: compare SMOTE, <code>class_weight="balanced"</code> , and threshold tuning — document the business trade-off. Train XGBoost with full MLflow experiment tracking: log all hyperparameters, metrics (precision, recall, F1, AUC-ROC), confusion matrix, and SHAP feature importance as artifacts.
Key Takeaways	- Accuracy is a misleading metric for fraud detection — a model that never detects fraud is 99.97% accurate. Precision-recall framing is mandatory. - Every experiment decision must be logged in MLflow: if the risk team asks why you chose threshold 0.35 over 0.50, you must show the data. - SHAP values for feature importance are the standard for model explainability to regulators under fair lending and credit risk governance frameworks.

Sample Session 2 — Auditable Loan Default Prediction with Hyperparameter Tuning

Business Scenario	An NBFC needs a loan default prediction model with fully documented, reproducible experiments for RBI audit. Every hyperparameter choice must be traceable to a logged experiment, and the final model must come with a Model Card.
What Participants Build	LightGBM model on a home loan dataset. Optuna hyperparameter optimisation with 50 trials — every trial logged in MLflow with parameters and validation AUC. Produce a Model Card documenting: training data characteristics, evaluation metrics, known limitations, and intended use. Package the model with joblib and wrap it in a FastAPI endpoint with SHAP-based explanation generation per prediction — the API returns the top 3 factors influencing the decision.
Key Takeaways	- MLflow as an audit trail: regulators and risk teams expect to see all trials, not just the winning one. - Model Cards are increasingly a regulatory artifact in credit risk AI — not optional documentation. - SHAP-based credit decision explanations are required under emerging AI governance frameworks and fair lending obligations.

Module Assignment — Insurance Motor Claims Fraud Detection System

Business Scenario	A general insurer estimates 8–12% of motor claims involve fraud (inflated repair costs, staged accidents, false theft). A deployed ML system with explainable outputs is required to assist fraud investigators — not replace them.
Deliverables	• Feature engineering on a provided claims dataset: claim amount-to-insured-value ratio, time since policy inception, claim frequency per policyholder, workshop risk score, vehicle age vs. claimed damage severity. • Train and compare three models: Logistic Regression (baseline), Random Forest, XGBoost with Optuna tuning. • Handle class imbalance with documented approach and justification. • Full MLflow tracking: every run with parameters, metrics, confusion matrix, and SHAP plots logged as artifacts. • FastAPI serving endpoint for the best model — input: claim features; output: fraud probability, SHAP top-3 factors, recommended action (Flag/Review/Pass). • Evaluation report: Precision, Recall, F1, AUC at thresholds 0.3, 0.5, 0.7 with business recommendation for optimal threshold.
Tools & Stack	Python, scikit-learn, XGBoost, LightGBM, Optuna, MLflow, SHAP, FastAPI, pandas, imbalanced-learn
Evaluation Criteria	• Feature engineering quality and domain relevance (30%) • MLflow tracking completeness — all runs, params, metrics, and artifacts (20%) • Model performance and threshold analysis with business recommendation (25%) • FastAPI endpoint with SHAP explanations (15%) • Code quality and reproducibility (10%)

Module 3: Deep Learning & Neural Network Engineering

This module moves participants from using ML libraries to building and training neural networks from first principles in PyTorch. In BFSI and Retail, deep learning powers document classification, transaction anomaly detection, and NLP complaint routing. Participants learn production training discipline: GPU memory management, mixed-precision training, checkpointing, and ONNX export for inference infrastructure.

Sample Session 1 — Banking Complaint Routing with DistilBERT in PyTorch

Business Scenario	A large bank receives 5,000+ written customer complaints daily. Manual triage to the correct department (Fraud, Credit Cards, Mortgages, Digital Banking, Branch Services) takes a 6-person team 3 hours each morning. An automated routing model reduces this to real-time.
What Participants Build	Custom PyTorch Dataset and DataLoader for CFPB (Consumer Financial Protection Bureau) consumer complaint data. Fine-tune DistilBERT with a 5-class classification head. Write the full training loop from scratch: forward pass, loss computation, backward pass, gradient clipping, scheduler step. Mixed-precision training with torch.cuda.amp (FP16) and gradient scaling. Checkpoint every epoch: save model state dict, optimiser state, epoch number — implement load-from-checkpoint for resuming. Evaluate with macro F1 and per-class precision/recall to identify weak departments.
Key Takeaways	- Writing a production training loop in PyTorch (not relying on Trainer APIs) is a fundamental skill for customisation and debugging in enterprise environments. - Mixed-precision training halves GPU memory usage and increases throughput by 30–50% with negligible accuracy loss — critical when GPU hours are metered. - Per-class metrics reveal where the model fails by department — aggregate F1 hides class-specific weaknesses that affect specific business units.

Sample Session 2 — Transaction Sequence Anomaly Detection: LSTM vs. Transformer

Business Scenario	Credit card issuers need to detect unusual spending sequences — small test purchases followed by large international transactions — that indicate account takeover. Sequence modelling is required, but the choice of architecture has significant implications for latency, memory, and accuracy.
What Participants Build	Encode transaction sequences as time-series features. Build an LSTM autoencoder for anomaly detection using reconstruction error as the anomaly score. Build a Temporal Fusion Transformer (TFT) for the same task. Train both, compare: training time, GPU memory footprint, AUC-ROC on a labelled anomaly dataset, inference latency. Develop a decision framework for when to use each architecture in production (sequence length, data volume, latency requirements).
Key Takeaways	- LSTM remains competitive for shorter sequences and lower-resource environments; Transformers win on long sequences but require more memory and data. - Reconstruction error as an anomaly score is an unsupervised approach that works when labelled fraud data is scarce — common in real deployments. - Architecture selection must be driven by production constraints (latency SLA, GPU budget, sequence length) — not just benchmark accuracy.

Module Assignment — Automated Document Classifier for Retail Loan Applications

Business Scenario	A retail bank receives 500+ loan applications daily, each containing multiple supporting documents: salary slips, bank statements, ITR filings, property papers, and guarantor documents. Staff currently spend 45 minutes per application manually identifying and organising document types before processing begins.
Deliverables	• Custom PyTorch Dataset class for 7-class document classification (Salary Slip, Bank Statement, ITR, Property Document, ID Proof, Form 16, Other). • Train a DistilBERT-based text classifier on extracted document text. • Train a ResNet-18-based image classifier on document page images. • Ensemble the two models (learned weighted average of softmax outputs) for final classification. • Mixed-precision training with gradient clipping; checkpoint every epoch; implement resume-from-checkpoint. • ONNX export of the best model; verify ONNX inference output matches PyTorch output. • Evaluation: per-class precision, recall, F1; confusion matrix with failure case analysis identifying the two most commonly confused document types.
Tools & Stack	Python, PyTorch, HuggingFace Transformers, torchvision, ONNX, scikit-learn, Weights & Biases
Evaluation Criteria	• Model architecture implementation quality for both classifiers (25%) • Training loop completeness: mixed precision, gradient clipping, checkpointing (25%) • Ensemble implementation and performance improvement over individual models (20%) • ONNX export and inference verification (10%) • Evaluation depth: confusion matrix and failure case analysis (20%)

Module 4: LLM Engineering — Working with Foundation Models

This module teaches participants to work with commercial LLMs as production engineering components — not chatbot wrappers. In BFSI, LLMs are deployed for document extraction, advisory assistants, regulatory interpretation, and report generation. The discipline here is cost-aware, latency-conscious, structured-output engineering with regression testing built in from day one.

Sample Session 1 — Structured Data Extraction from Financial Documents using Function Calling

Business Scenario	A wealth management firm processes 200+ fund fact sheets, investment reports, and prospectuses daily to update their internal database. Manual extraction takes 3 person-days. Key fields: fund name, ISIN code, NAV, expense ratio, asset allocation breakdown, top 10 holdings, risk rating, investment objective.
What Participants Build	Define a strict JSON schema for fund data extraction and implement using OpenAI function calling / Anthropic tool use to guarantee schema compliance. Handle long documents (up to 80 pages) using sliding window chunking: extract per-chunk, merge and deduplicate results. Compare extraction accuracy across GPT-4o, Claude Sonnet 4.5, and Gemini 2.5 Pro on 20 labeled documents — document cost and accuracy for each. Build a regression test suite: 20 labeled fund documents with expected outputs; run on every code commit.
Key Takeaways	- Function calling guarantees schema compliance without brittle regex parsing — the output is always a valid JSON object, never free text. - Multi-provider accuracy benchmarking prevents vendor lock-in decisions based on assumptions rather than evidence. - A regression test suite is non-negotiable for LLM pipelines in production — silent model updates can break extraction accuracy without warning.

Sample Session 2 — Multi-Provider LLM Router for a Banking Virtual Assistant

Business Scenario	A bank's virtual assistant handles 100,000 daily queries at a monthly LLM cost of ₹12 lakhs using a single premium model. Intelligent routing based on query complexity can reduce costs by 40–60% while maintaining quality on complex queries.
What Participants Build	Query complexity classifier (rule-based + fast LLM call) that routes: simple balance and transaction FAQs → GPT-4o-mini / Claude Haiku (cost: ~\$0.0003/query); regulatory and complex planning queries → GPT-4o / Claude Sonnet (cost: ~\$0.015/query); sensitive PII-bearing queries → on-premise model (privacy). Track cost per query, per query type, per day. Build a monthly cost projection dashboard. Implement streaming responses using SSE for real-time UX. Implement retry/fallback: if primary model fails, downgrade to cheaper model and flag for review.
Key Takeaways	- Routing logic should be grounded in measurable cost-per-query data — not intuition. Build the measurement before the router. - Streaming responses are essential in customer-facing BFSI applications: 3-second waits cause drop-offs; streamed tokens feel instantaneous. - Graceful degradation (fallback to cheaper model on failure) is a reliability requirement, not a nice-to-have.

Module Assignment — Insurance Policy Analysis and Comparison Pipeline

Business Scenario	An insurance broker receives a request to compare 5 health insurance policies for a corporate client. Each policy document is 40–80 pages. The broker needs a structured comparison report covering coverage limits, exclusions, waiting periods, cashless hospital networks, and premium structures within 30 minutes of the client request.
Deliverables	• Document ingestion pipeline handling PDFs of 5 to 80 pages; chunk appropriately for context window limits. • Structured extraction using function calling: coverage limits, exclusions list, waiting periods (per condition), network hospital count, premium structure, claim settlement ratio. • Multi-provider router: complex policy analysis → Claude Sonnet / GPT-4o; simple FAQ queries about the policy → GPT-4o-mini; log routing decisions. • Token budget tracking: total tokens per document, estimated cost, flag if processing exceeds \$0.10/document. • Automated regression test suite: 15 labeled policy documents with expected extraction outputs; run on every code change; report precision and recall per field. • Streaming generation of the final comparison report in markdown format.
Tools & Stack	Python, OpenAI / Anthropic / Gemini SDKs, FastAPI, LangChain (for streaming), pdfplumber, pytest
Evaluation Criteria	• Extraction accuracy on regression test set (field-level precision/recall) (35%) • Router implementation with logged routing decisions (20%) • Token/cost tracking accuracy (15%) • Regression test suite completeness and automation (20%) • Streaming response implementation (10%)

Module 5: RAG Systems — Build and Optimise End-to-End

RAG (Retrieval-Augmented Generation) is the dominant architecture for enterprise AI in BFSI. Regulatory Q&A, policy document assistants, product knowledge bases, and compliance tools all run on RAG. This module teaches participants to build RAG systems that are accurate, grounded, and measurably evaluated — because an RAG system that hallucinates regulatory guidance is worse than no system at all.

Sample Session 1 — Regulatory Document Q&A System with Hybrid Search

Business Scenario	Compliance officers at a mid-size bank spend 4–6 hours per week manually searching through RBI Master Circulars, SEBI regulations, IRDAI guidelines, and internal policies to answer specific queries from business units. Incorrect interpretations carry regulatory penalties and financial risk.
What Participants Build	Ingest 40+ real regulatory documents. Implement semantic chunking that preserves regulatory section boundaries (unlike fixed-size chunking which splits sections mid-sentence). Embed with BGE-M3 (strong performance on Indian financial and legal terminology). Implement hybrid search: BM25 for exact section number and regulation code lookups + dense vector for semantic queries, merged using Reciprocal Rank Fusion. Add a cross-encoder reranker (BGE-Reranker) to re-score the top-10 retrieved chunks. Generate citation-grounded answers that link back to the exact regulatory section — ungrounded answers are a compliance liability.
Key Takeaways	- Semantic chunking outperforms fixed-size chunking for legal text: regulatory sections must be kept intact to preserve meaning and prevent citation errors. - Hybrid search handles the full query spectrum: compliance officers search for both exact regulation codes (BM25 strength) and conceptual questions (dense vector strength). - Grounded citations are mandatory in compliance RAG — every answer must link to a specific regulation section to be usable by a compliance officer.

Sample Session 2 — RAG Evaluation with RAGAS — Iterative Quality Improvement

Business Scenario	A regulatory RAG system that gives confidently wrong answers is a systemic risk. The development process must include rigorous, measurable evaluation at every stage — not just at the end. Teams need to know when their changes improve or degrade retrieval and generation quality.
What Participants Build	Build a RAGAS evaluation pipeline for the regulatory Q&A system. Metrics: faithfulness (is the answer grounded in retrieved context?), answer relevance (does the answer actually address the question?), context recall (were the right sections retrieved?), context precision (were all retrieved sections relevant?). Run on a 50-question evaluation set with ground truth. Identify top failure modes: wrong chunks retrieved, insufficient context window, answer drift from context. Implement one improvement (e.g., change chunking strategy from fixed to semantic), re-run RAGAS, compare deltas.
Key Takeaways	- RAGAS is a development tool, not a final validation report — running it after every change is how you know you are improving, not degrading. - Faithfulness and answer relevance can diverge: a highly faithful answer may not actually answer the question if the wrong chunks were retrieved. - Building an automated eval pipeline that runs on every code change is the LLM equivalent of a CI test suite — non-negotiable in production.

Module Assignment — Retail Banking Product Knowledge Base RAG System

Business Scenario	A retail bank wants to reduce contact centre call volume by 30% by deploying a self-serve AI assistant for product queries (savings accounts, fixed deposits, credit cards, home loans, insurance). The assistant must give accurate, grounded answers — hallucinating interest rates or fee structures is a consumer protection risk.
Deliverables	- Ingest 15+ documents: product brochures, rate cards, terms and conditions, FAQs, service charges schedule — across formats (PDF, HTML, DOCX). - Implement and benchmark three chunking strategies: fixed-size, semantic, and section-aware (document structure-based). Document quality trade-offs. - Embed with two models: OpenAI text-embedding-3-small and BGE-M3. Benchmark retrieval precision@5 on a 30-question test set. - Implement hybrid search: BM25 + dense vector + Reciprocal Rank Fusion. - Add cross-encoder reranker (Cohere Rerank or BGE-Reranker); measure improvement in precision@5. - RAGAS evaluation: target faithfulness > 0.85, answer relevance > 0.80. If not met, iterate with documented improvement steps. - Document top 5 failure cases with root cause analysis and the fix applied.
Tools & Stack	Python, LangChain / LlamaIndex, Pinecone or Qdrant, BM25 (rank_bm25), BGE-M3, Cohere Rerank, RAGAS, FastAPI
Evaluation Criteria	- Chunking strategy benchmarking with documented quality trade-offs (20%) - Embedding model comparison with retrieval precision@5 data (20%) - Hybrid search and reranking implementation (20%) - RAGAS scores achieved and iteration documentation (25%) - Failure case analysis depth and quality of fixes (15%)

Module 6: Fine-Tuning, Alignment & Model Customisation

Generic foundation models use generic language. BFSI demands domain-specific terminology, compliance-mandated response patterns, and rejection of requests that violate regulatory constraints. This module teaches participants to customise models through supervised fine-tuning and alignment techniques — and, critically, when fine-tuning is the right choice versus better prompting or RAG.

Sample Session 1 — Fine-Tuning for BFSI Communication Standards using LoRA

Business Scenario	A bank's AI-powered customer communication tool uses a generic LLM that consistently fails domain standards: it says "money" instead of "funds," doesn't recognise RBI-mandated response templates, fails to decline requests for specific investment advice without disclaimers, and sounds unprofessional. Prompting improvements have plateaued.
What Participants Build	Curate a fine-tuning dataset of 1,000 BFSI customer service conversation examples: 100 from redacted real customer service logs (anonymised) and 900 synthetically generated using a powerful LLM. Apply quality filters: remove factually incorrect statements (verified against product docs), non-compliant responses (RBI template check), and repetitive or degenerate examples (deduplication by semantic similarity). Fine-tune Phi-3 Mini (3.8B) using LoRA (rank 16, alpha 32, target modules: q_proj, v_proj) with HuggingFace TRL. Monitor training and validation loss for overfitting. Evaluate fine-tuned vs. baseline on: terminology accuracy, compliance with response templates, and refusal rate for inappropriate requests.
Key Takeaways	- Quality over quantity: 1,000 high-quality, domain-filtered examples outperform 10,000 noisy ones — data curation is 60% of fine-tuning work. - LoRA makes fine-tuning a 3.8B model feasible on a single A100 40GB GPU in under 2 hours — adapter-based approaches are the standard for enterprise fine-tuning. - Measuring fine-tuning ROI vs. better prompting requires a benchmark: if the best prompt achieves 78% compliance and fine-tuning achieves 91%, the investment is justified.

Sample Session 2 — Synthetic Data Generation and DPO Alignment for Insurance Agent Support

Business Scenario	A life insurance company wants to fine-tune a model to support field agents with policy queries, claim guidance, and beneficiary change instructions. Real agent-customer conversations contain PII and cannot be used for training. Synthetic data generation is the only viable path.
What Participants Build	Design a prompt engineering pipeline to generate 2,000 diverse, realistic agent-customer conversations covering 12 scenario categories (new policy query, premium payment issue, claim initiation, beneficiary change, policy surrender, etc.). Apply quality filters: factual accuracy verification (does the generated response align with policy documents?), length filter, deduplication by TF-IDF similarity, and a compliance check (no unauthorised promises, no medical advice). Build a DPO preference dataset: for 300 scenarios, generate two responses — one compliant and professional (preferred), one casual or non-compliant (rejected). Fine-tune with DPO to align the model toward compliant responses.
Key Takeaways	- Synthetic data generation is the solution for PII-sensitive domains — designing the generation prompt is as important as the fine-tuning itself. - DPO is a lightweight alignment technique that does not require reward model training — the preferred/rejected pair format is easy to generate for compliance use cases.

- Quality filtering synthetic data requires domain knowledge — the filters must encode what "correct" looks like in the specific BFSI context.

Module Assignment — Health Insurance Claims Intake Fine-Tuned Model

Business Scenario	A health insurer's claims intake chatbot uses a generic LLM that frequently asks for the wrong documents, misuses medical terminology ("hospital stay" instead of "inpatient hospitalisation"), and occasionally promises claim settlement timelines it cannot guarantee — a compliance violation. The claims processing team has escalated this as a reputational risk.
Deliverables	• Generate 600+ synthetic health insurance claims intake conversations covering 10 scenario types: planned surgery, emergency hospitalisation, maternity, day-care procedure, pre-and-post hospitalisation, reimbursement claim, cashless claim, critical illness, OPD, and dental. • Quality filter pipeline: factual accuracy check (correct documents required per scenario), completeness check (all required intake fields collected), compliance check (no settlement timeline promises). Document rejection rate and reasons. • Fine-tune Phi-3 Mini with QLoRA (4-bit quantisation) on the filtered dataset. • Generate 300 DPO preference pairs (compliant professional response preferred; casual or non-compliant response rejected). Apply DPO fine-tuning on top of SFT model. • LLM-as-Judge evaluation on 50 held-out scenarios: score 1–5 on protocol adherence, information completeness, professionalism, and compliance. Compare SFT vs. SFT+DPO vs. baseline.
Tools & Stack	Python, HuggingFace TRL, PEFT, Unsloth, bitsandbytes (4-bit), OpenAI API (for generation + judging)
Evaluation Criteria	• Synthetic data quality: diversity across scenario types, quality filter rigor, documented rejection analysis (25%) • SFT fine-tuning implementation correctness (QLoRA, training loop, loss monitoring) (25%) • DPO dataset quality and alignment implementation (20%) • LLM-as-Judge evaluation rigor: scoring rubric quality, model comparison analysis (30%)

Module 7: AI Agent Engineering

Agentic AI systems are the frontier of BFSI automation — loan processing, KYC, claims investigation, and compliance monitoring are all candidate workflows. This module teaches participants to build production agents that are stateful, auditable, interruptible, and safe: properties that are not optional in regulated environments. LangGraph is the primary framework due to its support for cyclical workflows, checkpointing, and human-in-the-loop nodes.

Sample Session 1 — Loan Application Processing Agent with LangGraph

Business Scenario	A retail bank's loan processing involves 12+ manual steps across 4 teams. Average turn-around time is 5 business days. The bank targets automated straight-through processing for 70% of applications, with human review for borderline cases. Agents that cannot be paused, audited, or interrupted are not deployable in this context.
What Participants Build	LangGraph stateful agent with the following tools: DocumentParser (extracts income, liabilities, employment from uploaded docs using multimodal LLM call), CreditBureauLookup (mock CIBIL API with score and derogatory flags), DTICalculator (computes debt-to-income ratio), EligibilityEngine (applies bank's rule-based eligibility matrix), DecisionRouter (approve / reject / escalate to human). Conditional edges: credit score 650–750 → human-in-the-loop node; credit score > 750 and DTI < 40% → auto-approve. State checkpointing using PostgreSQL: agent state persisted at every node — if the service restarts, processing resumes from last checkpoint. Structured audit log of every tool call: input, output, timestamp, decision rationale.
Key Takeaways	- Checkpointing is essential in production: long-running agentic workflows (15+ seconds) must be resumable after service interruptions — without it, a restart restarts the entire process. - Human-in-the-loop nodes are how regulated agents handle the "grey zone" — automated decisions must have defined escalation thresholds, not binary approve/reject. - Every tool call must be logged with input and output for regulatory auditability — the audit log is as important as the decision itself.

Sample Session 2 — Multi-Agent Insurance Claims Processing System

Business Scenario	A general insurer's claims processing involves parallel workstreams — medical assessment, fraud check, and policy verification — that run sequentially today, creating a 10-day average settlement TAT. A multi-agent architecture enables parallel processing and reduces TAT to under 48 hours.
What Participants Build	Multi-agent system using Langflow or AutoGen with five specialised agents: IntakeAgent (extracts claim details from submitted form and documents), FraudDetectionAgent (checks claim patterns against historical fraud signals), MedicalAssessmentAgent (validates medical bills against treatment codes and market rates), PolicyVerificationAgent (confirms the claim falls within the policy's coverage scope and sub-limits), SettlementAgent (calculates final payable amount and generates settlement letter). Shared memory for inter-agent context passing. Parallel execution of FraudDetection + MedicalAssessment + PolicyVerification. Error handling: if FraudDetectionAgent times out, settlement is paused and human-flagged.
Key Takeaways	- Multi-agent architecture enables true parallelism for independent workstreams — when the three validation agents run in parallel rather than sequentially, TAT reduces by up to 60%. - Shared memory and handoff protocols must be explicitly designed — agents that "speak different data structures" cause silent failures at handoff points.

- Partial failure handling is mandatory: in a 5-agent pipeline, you must define what happens when any single agent fails — halt, skip, escalate, or retry.

Module Assignment — KYC Processing Agent System

Business Scenario	A bank onboards 500 new customers daily. The KYC process involves document collection, identity verification, sanctions screening, risk assessment, and routing for enhanced due diligence or approval. Manually, this takes 2–3 days per customer. The target is a 30-minute automated pipeline with human review for high-risk cases.
Deliverables	• LangGraph agent with tools: DocumentClassifier (identify document type: Aadhaar/PAN/Passport/Utility Bill), FieldExtractor (call multimodal LLM to extract name, DOB, ID number from document image), NameMatcher (fuzzy match extracted name vs. application name — flag mismatch > 15% edit distance), SanctionsChecker (mock OFAC/UN sanctions list API call), RiskScorer (PEP flag, high-risk country, occupation risk matrix → Low/Medium/High), DecisionRouter (Low → auto-approve; Medium → Enhanced Due Diligence queue; High → manual review queue). • Human-in-the-loop node for Medium and High risk cases with structured review form. • State checkpointing: PostgreSQL-backed persistence of agent state at every node. • Complete audit trail: JSON log of every tool call with input parameters, output, timestamp, and agent node name. • Test with 15 synthetic KYC cases: 5 clean low-risk, 5 medium-risk edge cases (name mismatch, expired document), 5 high-risk (PEP match, sanctions flag, high-risk country). • Graceful error handling: if any tool call fails after 2 retries, the case is escalated to manual review with error context logged.
Tools & Stack	Python, LangGraph, OpenAI/Anthropic (multimodal), PostgreSQL (checkpointing), FastAPI, pytest
Evaluation Criteria	• Agent architecture: tool design quality and decision routing logic (30%) • Human-in-the-loop implementation with structured review workflow (15%) • Audit trail completeness: all required fields present for every tool call (20%) • Test case coverage and quality of edge case handling (20%) • Error handling and graceful degradation (15%)

Module 8: Multimodal AI Engineering

Documents, images, audio, and forms are the primary data types in BFSI and Retail — not text-only inputs. Insurance claims arrive as photos of damage and handwritten forms. Banking calls are audio recordings. Retail invoices are scanned PDFs. This module equips participants to build multimodal pipelines that process real enterprise data types using vision-language models, Whisper, and document AI techniques.

Sample Session 1 — Insurance Claims Document Processing Pipeline

Business Scenario	A motor insurance company processes 800+ claims daily. Each claim involves: photos of vehicle damage, a PDF repair estimate from the workshop, a police report (often handwritten), and medical bills if there are injuries. Manually processing a single claim packet takes 3–4 hours. The target is under 5 minutes with AI-powered extraction and cross-document validation.
What Participants Build	End-to-end multimodal pipeline: Damage photo(s) → GPT-4o / Gemini 2.5 vision analysis (structured JSON: affected parts list, damage severity score 1–10, estimated repair category, visible fraud indicators). PDF repair estimate → pdfplumber extraction + LLM structured parsing (workshop name, GSTIN, repair line items, labour charges, parts charges, grand total). Cross-document validation: does claimed repair amount fall within $\pm 20\%$ of damage severity estimate? Flag anomalies. Output: unified structured JSON per claim with confidence scores per field. Build and evaluate on 50 real-format synthetic claim documents with ground truth.
Key Takeaways	- Cross-document consistency checking is where multimodal AI adds unique value in claims — a damage photo saying "minor scratch" vs. a repair estimate for ₹2 lakhs is a detectable fraud signal. - Confidence scores per extracted field are mandatory when automating decisions involving money: low-confidence fields must trigger human review, not automated processing. - Building on 50 labeled test documents before production deployment establishes a baseline accuracy that can be monitored and defended to operations teams.

Sample Session 2 — Banking Call Centre Transcription and Compliance Analysis

Business Scenario	A bank's contact centre handles 20,000 daily customer calls. Currently, QA analysts manually review 2% of calls for compliance violations and customer sentiment. An AI pipeline that analyses 100% of calls would identify compliance risks in near-real-time and surface coaching opportunities for agents.
What Participants Build	Whisper-based transcription pipeline optimised for banking call audio: custom vocabulary for financial terms, diarisation (speaker separation) so agent and customer turns are identified. Post-transcription LLM analysis pipeline: sentiment trajectory scoring (did customer sentiment improve or worsen over the call?), compliance flag detection (did the agent make any unauthorised settlement promises, guarantee timelines, or misrepresent product features?), issue categorisation from a predefined taxonomy, resolution confirmation detection. Batch processing architecture for 20K+ daily calls with cost management.
Key Takeaways	- Whisper is production-ready for banking call transcription: with custom vocabulary and fine-tuning for domain jargon, WER (Word Error Rate) falls below 8% on financial conversations. - LLM analysis of transcripts can detect compliance violations that keyword matching misses — context-aware understanding of agent statements is the differentiator. - Batch processing architecture for 20,000 calls requires careful cost management: Whisper + LLM analysis at full volume must be cost-modelled before deployment.

Module Assignment — Retail Supplier Invoice Processing System

Business Scenario	A large retail chain processes 10,000+ supplier invoices monthly across 500+ vendors. Current manual processing has a 3% error rate, a 15-day average processing time, and a team of 12 AP clerks. AI-powered extraction targets 99%+ field-level accuracy and same-day processing.
Deliverables	• Multimodal pipeline accepting invoice images (PNG, JPG) and PDFs in a single unified API endpoint. • Structured extraction: vendor name, vendor GSTIN, invoice number, invoice date, PO reference number, line items (description, HSN code, quantity, unit price, line total), subtotal, CGST/SGST/IGST amounts, grand total. • Handle four invoice format categories: structured printed (ERP-generated), semi-structured, handwritten, and multi-page (up to 10 pages). • Confidence score per extracted field (high/medium/low); fields with low confidence flagged for human verification. • Anomaly detection rules: duplicate invoice number check, amount > ₹5L without PO reference flag, GSTIN format validation (15-character alphanumeric), invoice date > 30 days old flag. • Accuracy evaluation: field-level precision and recall on a 60-invoice labeled test set across all four format categories; produce a format-breakdown performance matrix.
Tools & Stack	Python, GPT-4o / Gemini 2.5 (vision), pdfplumber, FastAPI, PostgreSQL (deduplication), pytest
Evaluation Criteria	• Extraction accuracy on 60-invoice test set: field-level precision and recall by format category (40%) • Format coverage: handles all four invoice format types robustly (20%) • Anomaly detection rules implementation and accuracy (20%) • Pipeline robustness: error handling, format-agnostic ingestion, confidence scoring (20%)

Module 9: LLMOps — Deploying and Operating AI in Production

Building an LLM application is 20% of the work. Deploying, monitoring, and operating it reliably in production — with BFSI-grade SLAs, cost governance, and audit trails — is the remaining 80%. This module teaches participants the full LLMOps stack: containerisation, Kubernetes deployment, observability, CI/CD with automated evaluation gates, semantic caching, and incident response. These are the skills that make the difference between a demo and a production system.

Sample Session 1 — Deploying a Compliance Advisory LLM with Full Observability

Business Scenario	A bank deploys an internal compliance advisory assistant. With 500 daily users across 4 business units and strict SLAs (P95 response time < 3 seconds, 99.9% monthly uptime), informal deployment is not acceptable. The compliance team also requires a complete audit trail of every query and response.
What Participants Build	Deploy an LLM application using vLLM for high-throughput serving with continuous batching — demonstrate throughput improvement over single-request serving (target: 3× improvement at 50 concurrent users). Containerise with Docker (multi-stage build for minimal image size). Deploy to Kubernetes: Deployment with 3 replicas, ClusterIP Service, HPA (scale on CPU utilisation > 70%). Set up LangSmith for trace-level observability: every query retrieved context chunk(s), model response, latency (TTFT + total), and token count logged with session ID and user department tag. Configure alerting: TTFT > 3s → PagerDuty alert; quality score < 0.70 → Slack notification to model owners; daily cost > \$500 → email to FinOps.
Key Takeaways	- vLLM's continuous batching enables 3–5× throughput improvement vs. serial inference at the same cost — understanding batching behaviour is essential for capacity planning. - Kubernetes HPA ensures the system scales under variable load: a Monday-morning spike at a bank is predictable and must not degrade response times. - Trace-level observability with session IDs is a compliance requirement: when a regulator asks "what did the AI tell the customer on this date?", the answer must be retrievable in under 5 minutes.

Sample Session 2 — CI/CD Pipeline, Semantic Caching, and A/B Testing for LLM Apps

Business Scenario	A production LLM assistant at a bank is updated weekly — new prompts, new models, new retrieved documents. Every update is a risk. The team needs automated quality gates that prevent regressions from reaching users, and cost optimisation for a system handling 100,000 daily queries.
What Participants Build	GitHub Actions CI/CD pipeline: on every pull request to main, automatically run a 50-query evaluation suite (golden test set with expected answers, evaluated by LLM-as-Judge). Block merge if quality score drops > 5% or P95 latency exceeds 3 seconds. Redis semantic cache: embed incoming queries and check cosine similarity against cached query embeddings; if similarity > 0.92, return cached response (simulate on 1,000 representative banking FAQ queries — measure cache hit rate and cost savings). A/B testing framework: route 10% of production traffic to new model/prompt version (Variant B), compare quality scores over 48 hours before full rollout. Prompt versioning using Git tags with release notes.
Key Takeaways	- CI/CD for LLMs prevents silent degradation: model updates, prompt changes, and retrieval changes all affect quality — automated gates catch regressions before users do. - Semantic caching achieves 35–45% cache hit rate for banking FAQ use cases, directly reducing monthly LLM API spend by a proportional amount.

- Controlled rollout (A/B testing) is the only safe way to change a production LLM system that serves real customers — big-bang updates are a support incident waiting to happen.

Module Assignment — Full LLMOps Setup for a Banking FAQ Assistant

Business Scenario	A bank's FAQ assistant goes live for 50,000 users across retail and corporate banking segments. You are the ML Platform engineer responsible for production readiness: the system must be observable, deployable, cost-efficient, and recoverable.
Deliverables	- Dockerfile (multi-stage build) and docker-compose for local end-to-end testing with all dependencies. - Kubernetes manifests: Deployment (3 replicas, resource limits: 2 CPU / 4Gi memory), ClusterIP Service, HPA (target CPU utilisation 70%, min 2 / max 8 replicas). - LangSmith observability integration: every trace logged with session ID, department tag, TTFT (ms), total latency (ms), input tokens, output tokens, estimated cost, and a quality score (LLM-as-Judge, 0–1 scale). - GitHub Actions CI pipeline: on PR to main, run 40 golden test queries; fail build if overall quality F1 drops > 5% or avg latency > 3 seconds; post results as PR comment. - Redis semantic cache: embed and cache frequent queries; implement and measure cache hit rate over 1,000 simulated queries; calculate monthly cost savings at current query volume. - Grafana dashboard (or LangSmith dashboard): daily query volume, avg TTFT, P95 latency, daily cost trend, quality score trend — 30-day rolling window. - Incident response runbook (written document): four scenarios with detection, diagnosis, and remediation steps: (1) quality score drop, (2) latency spike, (3) cost spike, (4) upstream LLM API outage.
Tools & Stack	Docker, Kubernetes, GitHub Actions, LangSmith, Redis, Grafana, Prometheus, Python/FastAPI
Evaluation Criteria	- Docker and Kubernetes setup correctness and production-readiness (20%) - CI pipeline with evaluation gates: test quality, latency check, PR comment (25%) - LangSmith observability: all required trace fields present (20%) - Semantic cache implementation and measured hit rate (15%) - Runbook quality: all 4 scenarios with actionable steps (20%)

Module 10: AI Evaluation, Safety & Responsible Engineering

In BFSI and Retail, deploying an AI system without rigorous safety evaluation is not a development shortcut — it is a regulatory and reputational risk. Loan rejection models that discriminate, RAG systems that hallucinate regulatory guidance, and chatbots that can be manipulated into giving unauthorised advice are systemic exposures. This module teaches participants to build evaluation frameworks, conduct structured red-teaming, implement guardrails, and produce the Responsible AI artefacts that regulated industries require.

Sample Session 1 — Red-Teaming and Guardrail Implementation for a Financial Services LLM

Business Scenario	A bank's AI-powered loan advisory assistant is being evaluated for go-live. The compliance and legal teams require documented evidence of adversarial testing before sign-off. A model that gives unauthorised investment advice, leaks customer data, or can be manipulated into bypassing disclaimers is a regulatory violation.
What Participants Build	Structured red-teaming process — not random prompt testing. Define a threat model for a BFSI advisory LLM (5 threat categories: regulatory violation, PII leakage, unfair treatment, jailbreak, hallucinated facts). Generate 30+ adversarial test cases across all categories. Execute: attempt prompt injection attacks, attempts to elicit specific investment advice without disclaimers, attempts to access other customers' data through indirect questioning, attempts to bypass compliance scripts through roleplay framing. Document all findings: prompt used, model response, severity (Critical/High/Medium/Low), exploitability. Implement NeMo Guardrails: input rails (detect off-topic financial advice requests), output rails (mandatory disclaimer injection, PII masking, refusal templates for out-of-scope queries). Test guardrail effectiveness: % of adversarial inputs blocked.
Key Takeaways	- Red-teaming is a structured process with a defined threat model — random prompt testing finds obvious issues but misses systematic vulnerabilities. - Guardrails are a compliance requirement in BFSI AI, not optional polish — every guardrail trigger must be logged to demonstrate the system is actively enforcing compliance rules. - Severity classification (Critical/High/Medium/Low) enables prioritisation: Critical vulnerabilities (regulatory violations) must be resolved before go-live; Low vulnerabilities may be accepted with documented risk.

Sample Session 2 — Bias Testing and Hallucination Detection in BFSI AI

Business Scenario	A loan recommendation AI that produces different outputs for demographically identical profiles violates fair lending laws. A RAG system that confidently cites a regulatory rule that does not exist is a systemic risk. Both issues must be detected and measured before production — and monitored continuously after.
What Participants Build	Bias audit framework: generate 500 synthetic loan applicant profiles, varying only demographic proxies (applicant name — varied across gender/ethnicity-coded names, pin code — varied across high/low-income areas), while holding all financial attributes constant (income, credit score, employment, DTI ratio). Measure recommendation consistency (loan amount offered, interest rate, approval/rejection). Apply disparate impact analysis: Chi-square test for statistical significance of demographic divergence. Visualise. Hallucination detection pipeline: for every RAG response, run a groundedness check using an LLM judge that reads the retrieved context and the generated answer, then determines whether every claim in the answer is supported by the context. Measure hallucination rate on 100 diverse regulatory queries.

Key Takeaways

- Disparate impact in loan recommendations is not just an ethics issue — it is a legal liability under the Equal Credit Opportunity Act and equivalent frameworks. Statistical testing is the standard of evidence.
- Controlling for all financial variables while varying demographic proxies is the correct methodology for isolating AI-driven bias vs. legitimate risk-based differentiation.
- Hallucination rate is a production KPI, not just a pre-launch metric — monitoring it daily catches model or retrieval degradation before it creates customer harm.

Module Assignment — Full Safety Evaluation of a Loan Recommendation Chatbot**Business Scenario**

Before a loan recommendation chatbot goes live to 200,000 retail banking customers, the Risk and Compliance committee requires a full safety evaluation report with documented findings, risk ratings, mitigations, and a formal sign-off checklist. This report will be reviewed by the bank's Chief Risk Officer.

Deliverables

- Red-team report: 25+ adversarial test cases across 5 threat categories (regulatory violation, data leakage, demographic bias trigger, hallucination induction, jailbreak). Each case documented with: prompt, model response, severity level, exploitability assessment, and recommended mitigation.
- Guardrail implementation: NeMo Guardrails or custom input/output filter pipeline. Effectiveness test: what % of adversarial inputs are correctly blocked? What is the false positive rate on benign inputs?
- Bias audit: 300 synthetic applicant profiles with controlled demographic variation. Disparate impact analysis with Chi-square statistical test. Visualised results and interpretation.
- Hallucination detection: groundedness checker for all model outputs against source policy documents. Measured hallucination rate on 100 diverse queries. Identified top hallucination-prone topics.
- Responsible AI Assessment Report (3–5 pages): Executive summary, risk matrix (severity × likelihood for each finding), mitigation actions taken and remaining, residual risk assessment, and a pre-deployment sign-off checklist with 20+ checkpoints.

Tools & Stack

Python, NeMo Guardrails, OpenAI/Anthropic (judge LLM), pandas, scipy (Chi-square), matplotlib, Word/PDF (for report)

Evaluation Criteria

- Red-team coverage: threat model completeness and adversarial prompt quality (25%)
- Guardrail effectiveness: block rate on adversarial inputs and false positive rate on benign inputs (20%)
- Bias audit methodology: correct statistical approach, clear visualisation, valid interpretation (25%)
- Hallucination detection pipeline and measured rate (15%)
- Responsible AI Assessment Report quality: executive readability, risk matrix rigour, actionable mitigations (15%)

5. Participant Engagement & Progress Assurance

Ensuring participants are actively engaged, not just present, requires a multi-layered system of accountability. The following mechanisms are embedded throughout the programme delivery.

Mechanism	Description	Frequency
Live Coding Checkpoints	During guided builds, the instructor calls random participants to share screens and explain the code they have written. This cannot be bypassed and makes passive attendance visible immediately.	Every session
Module Exit Quizzes	A 10-question concept-check quiz delivered via the LMS at the end of each module. Score < 70% triggers a mandatory revision session before the participant advances.	After each module
Assignment Submission Gate	Participants must submit and receive a passing evaluation on each module assignment before accessing the next module's materials in the LMS. Prevents skipping without consequence.	After each module
Code Review Sessions	Assignment submissions are reviewed in a structured peer + instructor code review session. Participants must both give and receive feedback — builds the review culture required in enterprise teams.	Weekly
Progress Analytics Dashboard	Programme coordinators have access to a real-time LMS dashboard showing: assignment submission rate, quiz scores, video completion %, and time-on-platform per participant. Lagging participants are flagged for check-in.	Continuous
Instructor Office Hours	Two 1-hour open office hours sessions per week where participants can bring stuck points, code issues, or conceptual questions. Attendance is tracked; low-engagement participants are encouraged to attend.	Twice weekly
Capstone Project Timeline	Participants are assigned their capstone project at the very start of ILT. Groups of 3–4 are formed before Module 1 begins, enabling teams to progressively apply each module's skills to their project as the programme advances. After ILT sessions conclude, groups receive 1–2 weeks to finalise, integrate, and polish their end-to-end system. During this post-ILT period, progress is tracked via structured weekly submission reports reviewed by the instructor and programme coordinator — covering what has been completed, blockers encountered, and the plan for the remaining time. Groups that fall behind the expected milestone are flagged for additional mentoring support.	Assigned Day 1 of ILT; 1–2 weeks post-ILT
Shark Tank Presentation & Evaluation	Upon completing their project, each group presents to a panel of industry practitioners in a formal Shark Tank-style review. The panel asks probing questions on technical decisions, domain reasoning, and production-readiness. Projects are evaluated on five rubrics: (1) Technical Implementation — does the end-to-end system function correctly with real test data? (2) Domain Relevance & Business Impact — does the solution address a genuine BFSI/Retail problem and quantify the projected impact? (3) Code Quality & Production-Readiness — is the codebase deployable, structured, and tested? (4) Architecture & Design Justification — are key technical choices explained and defended with data or reasoning? (5) Presentation Clarity & Q&A Defence — can the team communicate their solution and withstand expert scrutiny? Each rubric is scored 1–5; a minimum overall score of 3.0 is required to pass the capstone.	Post-ILT (after 1–2 weeks)

6. Sample Capstone Projects

Participants work in groups of 3–4 on one capstone project selected from the options below. All projects are grounded in BFSI or Retail domain realities and require integrating skills from multiple modules. Projects are presented in a Shark Tank-style review where a panel of industry practitioners evaluates both the technical solution and its business viability.

Capstone 1 — Intelligent Motor Claims Processing Platform (Insurance)

Business Problem	A motor insurer processes 800 claims per day with a 10-day average settlement TAT and 12% suspected fraud rate. The manual pipeline is expensive and slow.
What to Build	An end-to-end AI platform: multimodal document ingestion (damage photos + repair estimates + medical bills) → structured data extraction → fraud detection (ML model) → policy coverage verification (RAG) → settlement recommendation (agentic workflow) → customer communication (LLM-generated letter). Full audit trail, human-in-the-loop for disputed cases.
Modules Integrated	Module 2 (Fraud ML), Module 5 (RAG for policy), Module 7 (Agent), Module 8 (Multimodal), Module 9 (LLMOps), Module 10 (Safety Eval)
Evaluation Criteria	- End-to-end functional pipeline with real-format test claims (30%) - Fraud detection model performance: AUC, precision at threshold (20%) - RAG accuracy: policy coverage Q&A faithfulness > 0.85 (20%) - Agent: audit trail completeness and human-in-the-loop correctness (15%) - Business impact quantification: projected TAT reduction and cost savings (15%)

Capstone 2 — Agentic Loan Origination System (Retail Banking)

Business Problem	A retail bank's loan processing takes 5 business days due to manual steps across 4 teams. Straight-through processing for low-risk profiles is not yet automated.
What to Build	Multi-agent LangGraph system: DocumentCollectionAgent → KYCVerificationAgent (multimodal ID extraction + sanctions check) → CreditAssessmentAgent (credit bureau + income verification) → RegulatoryComplianceAgent (RAG over RBI lending guidelines) → DecisionAgent (approve/reject/escalate with explainable rationale). Full checkpointing, human-in-the-loop for borderline cases, complete audit log for regulatory review.
Modules Integrated	Module 4 (LLM structured output), Module 5 (Regulatory RAG), Module 7 (LangGraph agents), Module 8 (Multimodal KYC), Module 9 (LLMOps deployment)
Evaluation Criteria	- Agent workflow completeness: all 5 agents functional with correct handoffs (35%) - KYC accuracy: field extraction precision on 20 test documents (20%) - RAG compliance check: correct regulatory citations (20%) - Audit log: all tool calls logged with required fields (15%) - Deployment: Dockerised with working API endpoint (10%)

Capstone 3 — Real-Time Fraud Explanation Engine (Financial Services)

Business Problem	A payment network's fraud detection model flags transactions but provides no explanation. Investigators spend 8 minutes per flagged transaction gathering context before deciding to block or release. Explanation generation must happen in under 10 seconds.
What to Build	Classical ML fraud detection model (XGBoost, trained on transaction data) integrated with an LLM explanation layer: when a transaction is flagged, an LLM agent gathers context (merchant history, customer spending pattern, device fingerprint signals), generates a human-readable investigation brief for the analyst, and suggests the top 3 investigation actions. Full LLMOps setup: vLLM serving, LangSmith observability, latency SLA monitoring.
Modules Integrated	Module 2 (Fraud ML), Module 4 (LLM structured output), Module 7 (Agent), Module 9 (LLMOps), Module 10 (Hallucination detection for explanation accuracy)
Evaluation Criteria	- Fraud model AUC and precision at business-optimal threshold (25%) - Explanation quality: LLM-as-Judge scores on accuracy, actionability, clarity (30%) - Latency: end-to-end (ML inference + LLM explanation) P95 < 10 seconds (20%) - LLMOps: observability dashboard showing latency, cost, quality trend (15%) - Hallucination rate on explanation outputs < 5% (10%)

Capstone 4 — Regulatory Intelligence Platform (BFSI Compliance)

Business Problem	Compliance officers at a BFSI firm spend 6+ hours weekly manually searching regulatory documents to answer business queries, track regulatory changes, and generate compliance checklists. An incorrect regulatory interpretation carries potential penalty exposure.
What to Build	RAG system over 60+ RBI, SEBI, and IRDAI regulatory documents with hybrid search and a fine-tuned embedding model (fine-tuned on BFSI regulatory text for better retrieval). Features: citation-grounded Q&A, regulatory change tracker (identify what changed between two versions of a circular), compliance checklist generator (given a business activity, generate the applicable regulatory checklist). Full RAGAS evaluation. Fine-tuned model for regulatory terminology.
Modules Integrated	Module 5 (RAG), Module 6 (Fine-tuning embedding model), Module 9 (LLMOps), Module 10 (Hallucination detection, safety evaluation)
Evaluation Criteria	- RAGAS scores: faithfulness > 0.88, answer relevance > 0.82 (30%) - Fine-tuned embedding model: retrieval improvement vs. generic embedding (25%) - Regulatory change tracker: correctly identifies changed/added/removed provisions (20%) - Hallucination rate < 3% on regulatory Q&A (15%) - UI/API completeness and usability (10%)

Capstone 5 — Retail Banking Intelligent Customer Assistant (Retail Banking)

Business Problem	A retail bank's contact centre receives 15,000 daily calls. 60% are routine queries (balance, transaction status, product information) that do not require a human agent. A safe, accurate AI assistant that handles routine queries and escalates complex ones would reduce call centre costs by 40%.
-------------------------	--

What to Build	Customer-facing AI assistant: RAG for product Q&A (interest rates, account types, loan products), agentic workflow for transactional queries (mock API tool calls for balance enquiry, last 5 transactions, FD maturity date), complaint intake with structured data capture and ticketing, and escalation to human agent with full context summary. Complete safety evaluation: red-teaming (20+ adversarial prompts), guardrails (no financial advice, no account modification), bias audit, hallucination rate measurement. Full LLMOps deployment.
Modules Integrated	Module 5 (RAG), Module 7 (Agent), Module 9 (LLMOps), Module 10 (Safety Eval — all components)
Evaluation Criteria	• RAG accuracy on product Q&A: faithfulness > 0.85 (20%) • Agent tool calling: correct responses to 15 transactional test queries (20%) • Safety evaluation: red-team report, guardrail effectiveness, hallucination rate (30%) • LLMOps: deployed with CI/CD, observability, semantic caching (20%) • Demo quality and business case articulation in Shark Tank (10%)

Capstone 6 — AI-Powered Insurance Underwriting Assistant (Insurance)

Business Problem	Insurance underwriters spend 2–3 hours per commercial policy application gathering data, assessing risk factors, and writing underwriting reports. Throughput is the bottleneck: a team of 10 underwriters processes 50 applications per day; the target is 150 applications per day with AI assistance.
What to Build	Multimodal underwriting pipeline: ingest application form (PDF/image) + supporting documents (financial statements, property photos, business registration) → extract structured applicant and risk data → enrich with external risk signals (mock API: industry loss ratios, location risk score) → RAG over underwriting guidelines for product-specific risk rules → generate a structured Underwriting Report (risk score, key risk factors with evidence, coverage recommendation, premium range, exclusion list). Bias audit on risk scoring. Human-in-the-loop review for borderline cases.
Modules Integrated	Module 4 (Structured LLM output), Module 5 (RAG for guidelines), Module 7 (Agent), Module 8 (Multimodal extraction), Module 10 (Bias audit)
Evaluation Criteria	• Extraction accuracy: field-level precision on 30 test applications (25%) • RAG guideline compliance: correct risk rules cited for 15 underwriting scenarios (25%) • Underwriting report quality: LLM-as-Judge scoring on completeness, accuracy, clarity (20%) • Bias audit: demographic proxy analysis on synthetic applicant profiles (20%) • Pipeline end-to-end latency: target < 5 minutes per application (10%)

7. Cloud Stack Cohort Variants (Delta View)

The programme can be delivered in three cloud-specific cohorts: AWS, Azure, and Google Cloud. The core curriculum — frameworks, architectural patterns, and domain scenarios — remains identical across all cohorts. The delta is in the cloud-native services used for specific infrastructure functions. The table below maps each infrastructure category to the relevant service for each cohort.

All cohorts use the same open-source frameworks throughout the programme: LangChain/LangGraph, LlamaIndex, HuggingFace, PyTorch, MLflow, RAGAS, NeMo Guardrails, FastAPI, and Docker. These are cloud-agnostic and industry-standard.

Infrastructure Category	AWS Cohort	Azure Cohort	GCP Cohort
LLM API Access	Amazon Bedrock(Claude, Llama, Titan, Mistral)	Azure OpenAI Service(GPT-4o, o1, DALL·E 3)	Vertex AI Model Garden(Gemini 2.5, Claude, Llama)
Vector Database (Managed)	Amazon OpenSearch Service(k-NN vector index)	Azure AI Search(vector index + hybrid search)	Vertex AI Vector Search(Matching Engine)
Vector DB (Self-Hosted)	Qdrant / pgvector on RDS(PostgreSQL + pgvector)	pgvector on Azure Databasefor PostgreSQL	AlloyDB with pgvector(Google-managed PostgreSQL)
Model Training	Amazon SageMaker Training(managed training jobs)	Azure Machine Learning(compute clusters)	Vertex AI Training(custom training jobs)
Model Serving / Inference	SageMaker Real-Time Endpoints+ SageMaker Serverless Inference	Azure ML Online Endpoints(managed + Kubernetes)	Vertex AI Prediction(dedicated + shared endpoints)
Object Storage	Amazon S3(datasets, model artifacts)	Azure Data Lake Storage Gen2+ Azure Blob Storage	Google Cloud Storage (GCS)(buckets for all artifacts)
Data Warehouse	Amazon Redshift(feature store backend)	Azure Synapse Analytics+ Azure SQL Database	Google BigQuery(feature data + analytics)
Experiment Tracking	MLflow on EC2 / SageMakerExperiments (native)	Azure ML Experiments+ MLflow (integrated)	Vertex AI Experiments+ MLflow on GCE
ML Pipeline Orchestration	Amazon MWAA (managed Airflow)+ AWS Step Functions	Azure Data Factory+ Azure Databricks Workflows	Cloud Composer (managed Airflow)+ Vertex AI Pipelines
Container Registry	Amazon ECR(Elastic Container Registry)	Azure Container Registry (ACR)	Google Artifact Registry(replaces Container Registry)
Kubernetes (managed)	Amazon EKS(Elastic Kubernetes Service)	Azure Kubernetes Service (AKS)	Google Kubernetes Engine (GKE)(Autopilot or Standard)
CI/CD	GitHub Actions + AWS CodePipeline+ AWS CodeBuild	GitHub Actions+ Azure DevOps Pipelines	GitHub Actions+ Google Cloud Build
Secrets Management	AWS Secrets Manager+ AWS Parameter Store	Azure Key Vault(secrets, keys, certificates)	Google Secret Manager(versioned secrets)

Semantic Cache (Redis)	Amazon ElastiCache for Redis (Serverless or cluster)	Azure Cache for Redis (Basic, Standard, Premium)	Memorystore for Redis (Basic or Standard tier)
Observability / Monitoring	Amazon CloudWatch+ AWS X-Ray (tracing)	Azure Monitor+ Application Insights	Cloud Monitoring+ Cloud Trace+ Cloud Logging
GPU Infrastructure	EC2 P3/P4/G5 instances+ SageMaker GPU training	Azure NC-series VMs (A100/H100)+ Azure ML GPU compute	GCE A2/A3 VMs (A100/H100)+ Vertex AI GPU training

What Remains the Same Across All Cohorts

- All 10 module curricula, domain scenarios, assignments, and capstone projects are identical — cloud services are infrastructure, not curriculum.
- Open-source frameworks: LangChain, LangGraph, LlamaIndex, HuggingFace TRL/PEFT, PyTorch, MLflow, RAGAS, NeMo Guardrails, FastAPI, Docker, vLLM.
- All BFSI and Retail domain datasets, problem statements, and business scenarios.
- Evaluation frameworks, assignment rubrics, and capstone project structure.
- Instructor delivery methodology, session structure, and engagement mechanisms.

8. Certifications, Assessment & Next Steps

Programme Component	Details
Pre-ILT Self-Paced	3 hours of LMS content covering Python, NumPy, linear algebra, APIs. Completion required before ILT access is granted.
Module Quizzes	10-question MCQ quiz after each module via LMS. Pass mark: 70%. Retake available once; below 70% on retake triggers revision session.
Module Assignments	Practical assignment for each of the 10 modules. Evaluated by the instructor on the rubric provided. Pass mark: 60%. Mandatory for progression.
Capstone Project	Groups of 3–4. Three milestone reviews. Final Shark Tank presentation to a panel of industry experts. Scored on technical quality, domain applicability, and business impact framing.
Certification	Issued by Incubity by Ambilio upon successful completion of all modules, all assignments, and the capstone. Verifiable digital credential.
Post-ILT Access	30 minutes of guided post-ILT self-paced content on LMS: curated resources, reference architectures, and practice problem sets for continued development.

For questions regarding programme scheduling, cohort sizing, instructor profiles, or customisation for specific EXL project domains, please contact the Ambilio programme team. We are committed to ensuring that every participant who completes this programme is deployable on client-facing AI engineering engagements from day one.

© 2026 Ambilio. All rights reserved. Confidential — prepared exclusively for EXL.